

```

mysql> -- TCL - Transaction Control Language
mysql> /* transaction : The process of combining all related operations into a
single unit and executing
/*> then according to the rule "either all operations should succeed or none
should succeed" is called transaction management.
/*> a transaction is considered a single unit of work and follows the principle
"either all or none";
/*>
/*>
/*> example: Funds transfer
/*> consider a money transfer process between two accounts:
/*> debit amount from the sender's account
/*> credit amount into the receiver's account
/*> both operations should be executed as a single transaction.
/*> If the amount is debited from the sender's account successfully, but
crediting the receiver's account fails, it may lead
/*> to data inconsistency problems.. */
mysql>
mysql>
mysql> /* two types of transactions
/*> 1.local transaction
/*> 2.global transaction
/*>
/*>
/*> 1.local transaction
/*> -----
/*> In local transaction all operations of a transaction are executed on the
same database.
/*> ex:
/*> funds transfer from one to another account where both accounts belong to
same bank..
/*>
/*>
/*>
/*> 2.global transaction
/*> -----
/*> In global transaction all operations of a transaction are executed on the
different databases.
/*> ex:
/*> funds transfer from one to another account where both accounts belong to
different banks..
/*>
/*>
/*>
/*> ACID properties
/*> -----
/*> Atomicity: all or nothing.
/*> Consistency: The database remains in a valid state before and after
transaction.
/*> Isolation: Transactions do not interfere with each other

```

```

/*> Ex: a receiver's account can get muplt sender money
/*> Durability: Committed changes are permanent.
/*>
/*>
/*> -- TCL - Transaction Control Language
/*> -----
/*> 1.commit
/*> -----
/*> commit is used to permanently save all changes made during the current
transaction into the database.
/*> once committed changes become permanent.
/*> other users can see the changes.
/*> you cannot undo the changes using rollback.
/*>
/*> 2.rollback
/*> -----
/*> rollback is used to undo all changes made in the current transaction before
a commit.
/*> if something goes wrong. rollback restores the database to its previous
state.
/*>
/*> */

```

```
mysql> select * from students;
```

student_id	name	age
1	Arjun	20
2	Bhavana	21
3	Charan	22
4	Divya	20
5	Eswar	23
6	Farhan	21
7	Gowri	22

```
7 rows in set (0.04 sec)
```

```
mysql> drop table students;
```

```
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> create table students(
-> std_id int primary key,
-> std_name varchar(50),
-> marks int);
```

```
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> INSERT INTO STUDENTS VALUES(111,'MALLI',97),(222,'MAHI',100);
```

```
Query OK, 2 rows affected (0.01 sec)
```

Records: 2 Duplicates: 0 Warnings: 0

mysql> COMMIT;

Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM STUDENTS;

std_id	std_name	marks
111	MALLI	97
222	MAHI	100

2 rows in set (0.00 sec)

mysql> START TRANSACTION;

Query OK, 0 rows affected (0.00 sec)

mysql> update students set marks=60 where std_id=111;

Query OK, 1 row affected (0.01 sec)

Rows matched: 1 Changed: 1 Warnings: 0

mysql> SELECT * FROM STUDENTS;

std_id	std_name	marks
111	MALLI	60
222	MAHI	100

2 rows in set (0.00 sec)

mysql> rollback;

Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM STUDENTS;

std_id	std_name	marks
111	MALLI	97
222	MAHI	100

2 rows in set (0.00 sec)

mysql> START TRANSACTION;

Query OK, 0 rows affected (0.00 sec)

mysql> delete from students where std_id=111;

Query OK, 1 row affected (0.01 sec)

mysql> commit;

Query OK, 0 rows affected (0.01 sec)

```
mysql> rollback;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> SELECT * FROM STUDENTS;
+-----+-----+-----+
| std_id | std_name | marks |
+-----+-----+-----+
| 222    | MAHI     | 100    |
+-----+-----+-----+
1 row in set (0.00 sec)
```

```
mysql>
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> INSERT INTO STUDENTS VALUES(333,'SUNNY',60),(444,'BUNNY',86);
Query OK, 2 rows affected (0.00 sec)
Records: 2 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM STUDENTS;
+-----+-----+-----+
| std_id | std_name | marks |
+-----+-----+-----+
| 222    | MAHI     | 100    |
| 333    | SUNNY    | 60     |
| 444    | BUNNY    | 86     |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> UPDATE STUDENTS SET STD_NAME ='MANI' WHERE STD_ID=222;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0
```

```
mysql> SELECT * FROM STUDENTS;
+-----+-----+-----+
| std_id | std_name | marks |
+-----+-----+-----+
| 222    | MANI     | 100    |
| 333    | SUNNY    | 60     |
| 444    | BUNNY    | 86     |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

```
mysql> DELETE FROM STUDENTS WHERE STD_ID=333;
Query OK, 1 row affected (0.00 sec)
```

```
mysql> SELECT * FROM STUDENTS;
```

std_id	std_name	marks
222	MANI	100
444	BUNNY	86

2 rows in set (0.00 sec)

```
mysql> ROLLBACK;
```

Query OK, 0 rows affected (0.01 sec)

```
mysql> SELECT * FROM STUDENTS;
```

std_id	std_name	marks
222	MAHI	100
333	SUNNY	60
444	BUNNY	86

3 rows in set (0.00 sec)

```
mysql> ROLLBACK;
```

Query OK, 0 rows affected (0.00 sec)

```
mysql> SELECT * FROM STUDENTS;
```

std_id	std_name	marks
222	MAHI	100
333	SUNNY	60
444	BUNNY	86

3 rows in set (0.00 sec)

```
mysql> START TRANSACTION;
```

Query OK, 0 rows affected (0.00 sec)

```
mysql> INSERT INTO STUDENTS VALUES(555,'MUNNY',60),(666,'DUNNY',86);
```

Query OK, 2 rows affected (0.00 sec)

Records: 2 Duplicates: 0 Warnings: 0

```
mysql> SELECT * FROM STUDENTS;
```

std_id	std_name	marks
222	MAHI	100
333	SUNNY	60
444	BUNNY	86
555	MUNNY	60

std_id	std_name	marks
666	DUNNY	86

5 rows in set (0.00 sec)

```
mysql> UPDATE STUDENTS SET STD_NAME ='MANI' WHERE STD_ID=222;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

```
mysql> SELECT * FROM STUDENTS;
```

std_id	std_name	marks
222	MANI	100
333	SUNNY	60
444	BUNNY	86
555	MUNNY	60
666	DUNNY	86

5 rows in set (0.00 sec)

```
mysql> SAVEPOINT SP1;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> DELETE FROM STUDENTS WHERE STD_ID=444;
Query OK, 1 row affected (0.00 sec)
```

```
mysql> SELECT * FROM STUDENTS;
```

std_id	std_name	marks
222	MANI	100
333	SUNNY	60
555	MUNNY	60
666	DUNNY	86

4 rows in set (0.00 sec)

```
mysql> ROLLBACK TO SP1;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> SELECT * FROM STUDENTS;
```

std_id	std_name	marks
222	MANI	100
333	SUNNY	60
444	BUNNY	86
555	MUNNY	60
666	DUNNY	86

5 rows in set (0.00 sec)

```
mysql> /* Introductionn to stored procedures
/*> -----
/*> a stored procedure is a named, precompiled collection of one or more sql
statements that is stored
/*> inside the database and can be executed called whenever required.
/*> instead of sending long block of sql from an application every time,the
application simply called the procedure by name.
/*>
/*>
/*> why use stored procedures?
/*> -----
/*> 1.Performance
/*> 2.Reusability
/*> 3.Security
/*> 4.Reduced network traffic
/*> 5.Maintainability
/*> 6.Consistency.
/*>
/*> Advantages
/*> -----
/*> faster execution after completion
/*> reduced duplicate sql.
/*> supports transaction,loops,conditions,variables...etc.
/*> easy maintainence...
/*>
/*>
/*> Disadvantages
/*> -----
/*> debugging can be difficult.
/*> vendor specific syntax.
/*> complex procedures may become hard to maintain.
/*>
/*> synrax to create stored procedures
/*> -----
/*> Delimiter //
/*> create procedure procedure_name()
/*> Begin
/*> --- sql statements
/*> End //
/*> Delimiter ;
/*> */
mysql>
mysql> /* In Parameter
/*> -----
/*> An IN parameter is used to pass a value into a stored procedure.
/*> the procedure can use the value, but changes made to the parameter inside
the
/*> procedure are not returned to the caller.
```

```

/*>
/*>
/*> syntax
/*> -----
/*> create procedure procudure_name(IN parameter_name datatype)
/*> Begin
/*>         sql statements
/*> End ;
/*> Delimiter ;
/*> */

```

```

mysql>
mysql>
mysql> CREATE DATABASE companydb;
Query OK, 1 row affected (0.04 sec)

```

```

mysql>
mysql> USE companydb;
Database changed

```

```

mysql>
mysql> CREATE TABLE employees (
->     emp_id INT PRIMARY KEY,
->     emp_name VARCHAR(50),
->     department VARCHAR(50),
->     salary DECIMAL(10,2),
->     experience INT,
->     city VARCHAR(50),
->     age INT
-> );
Query OK, 0 rows affected (0.08 sec)

```

```

mysql>
mysql> INSERT INTO employees
-> VALUES
-> (101, 'Ravi', 'IT', 45000, 2, 'Hyderabad', 24),
-> (102, 'Kiran', 'HR', 40000, 3, 'Vijayawada', 27),
-> (103, 'Suresh', 'IT', 55000, 5, 'Hyderabad', 30),
-> (104, 'Anil', 'Sales', 35000, 1, 'Chennai', 23),
-> (105, 'Priya', 'Finance', 60000, 6, 'Bangalore', 32),
-> (106, 'Sneha', 'IT', 50000, 4, 'Vizag', 28),
-> (107, 'Rahul', 'HR', 42000, 2, 'Hyderabad', 25),
-> (108, 'Arjun', 'Sales', 38000, 3, 'Chennai', 26),
-> (109, 'Divya', 'Finance', 65000, 7, 'Bangalore', 34),
-> (110, 'Manoj', 'IT', 70000, 8, 'Hyderabad', 35);
Query OK, 10 rows affected (0.01 sec)
Records: 10  Duplicates: 0  Warnings: 0

```

```

mysql> select * from employees;

```

emp_id	emp_name	department	salary	experience	city	age
101	Ravi	IT	45000	2	Hyderabad	24
102	Kiran	HR	40000	3	Vijayawada	27
103	Suresh	IT	55000	5	Hyderabad	30
104	Anil	Sales	35000	1	Chennai	23
105	Priya	Finance	60000	6	Bangalore	32
106	Sneha	IT	50000	4	Vizag	28
107	Rahul	HR	42000	2	Hyderabad	25
108	Arjun	Sales	38000	3	Chennai	26
109	Divya	Finance	65000	7	Bangalore	34
110	Manoj	IT	70000	8	Hyderabad	35

101	Ravi	IT	45000.00	2	Hyderabad	24
102	Kiran	HR	40000.00	3	Vijayawada	27
103	Suresh	IT	55000.00	5	Hyderabad	30
104	Anil	Sales	35000.00	1	Chennai	23
105	Priya	Finance	60000.00	6	Bangalore	32
106	Sneha	IT	50000.00	4	Vizag	28
107	Rahul	HR	42000.00	2	Hyderabad	25
108	Arjun	Sales	38000.00	3	Chennai	26
109	Divya	Finance	65000.00	7	Bangalore	34
110	Manoj	IT	70000.00	8	Hyderabad	35

10 rows in set (0.00 sec)

```
mysql> DELIMITER //
```

```
mysql> CREATE PROCEDURE getEmployeeById(IN eid INT)
```

```
-> BEGIN
```

```
->     SELECT * FROM EMPLOYEES WHERE EMP_ID=EID;
```

```
-> END //
```

Query OK, 0 rows affected (0.02 sec)

```
mysql> DELIMITER ;
```

```
mysql> CALL getEmployeeById(101);
```

emp_id	emp_name	department	salary	experience	city	age
101	Ravi	IT	45000.00	2	Hyderabad	24

1 row in set (0.01 sec)

Query OK, 0 rows affected (0.03 sec)

```
mysql> CALL getEmployeeById(110);
```

emp_id	emp_name	department	salary	experience	city	age
110	Manoj	IT	70000.00	8	Hyderabad	35

1 row in set (0.00 sec)

Query OK, 0 rows affected (0.02 sec)

```
mysql> CALL getEmployeeById(102);
```

emp_id	emp_name	department	salary	experience	city	age
102	Kiran	HR	40000.00	3	Vijayawada	27

1 row in set (0.00 sec)

Query OK, 0 rows affected (0.05 sec)

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE getEmployeeBysalary(IN eid INT)
-> BEGIN
->     SELECT salary from employees Where salary=eid;
-> END //
```

Query OK, 0 rows affected (0.01 sec)

```
mysql> DELIMITER ;
mysql> Call getEmployeeBysalary(102);
Empty set (0.01 sec)
```

Query OK, 0 rows affected (0.01 sec)

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE getEmployeeBysal(IN eno INT)
-> BEGIN
->     SELECT salary from employees Where emp_id=eno;
-> END //
```

Query OK, 0 rows affected (0.01 sec)

```
mysql> DELIMITER ;
mysql> Call getEmployeeBysal(105);
```

```
+-----+
| salary |
+-----+
| 60000.00 |
+-----+
```

1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)

```
mysql> DELIMITER ;
mysql> Call getEmployeeBysalary(40000);
```

```
+-----+
| salary |
+-----+
| 40000.00 |
+-----+
```

1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

```
mysql> Show procedure status
-> where db = database();
```

```
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
| Db      | Name      | Type      | Definer      | Modified    |
| Created | Security_type | Comment | character_set_client |
```

```

collation_connection | Database Collation |
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+
| companydb | getEmployeeById      | PROCEDURE | root@localhost | 2026-09-23
14:35:18 | 2026-09-23 14:35:18 | DEFINER   |                | cp850
cp850_general_ci | utf8mb4_0900_ai_ci |
| companydb | getEmployeeBysal     | PROCEDURE | root@localhost | 2026-09-23
14:40:28 | 2026-09-23 14:40:28 | DEFINER   |                | cp850
cp850_general_ci | utf8mb4_0900_ai_ci |
| companydb | getEmployeeBysalary  | PROCEDURE | root@localhost | 2026-09-23
14:38:14 | 2026-09-23 14:38:14 | DEFINER   |                | cp850
cp850_general_ci | utf8mb4_0900_ai_ci |
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+
3 rows in set (0.01 sec)

```

```

mysql> DELIMITER //
mysql> CREATE PROCEDURE getEmployeesBydepartment(IN dept varchar(20))
-> BEGIN
->     select * from employees where department=dept;
-> END //
Query OK, 0 rows affected (0.01 sec)

```

```

mysql> DELIMITER ;
mysql> call getEmployeesBydepartment('IT');
+-----+-----+-----+-----+-----+-----+-----+
| emp_id | emp_name | department | salary   | experience | city      | age |
+-----+-----+-----+-----+-----+-----+-----+
| 101    | Ravi     | IT         | 45000.00 | 2          | Hyderabad | 24  |
| 103    | Suresh   | IT         | 55000.00 | 5          | Hyderabad | 30  |
| 106    | Sneha    | IT         | 50000.00 | 4          | Vizag     | 28  |
| 110    | Manoj    | IT         | 70000.00 | 8          | Hyderabad | 35  |
+-----+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

```

Query OK, 0 rows affected (0.01 sec)

```

mysql> drop procedure getEmployeeBysalary;
Query OK, 0 rows affected (0.02 sec)

```

```

mysql> DELIMITER //
mysql> CREATE PROCEDURE getEmployeesByRange(IN min decimal(10,2) in max
decimal(10,2))
-> BEGIN
-> ^C
mysql>
mysql> DELIMITER //
mysql> CREATE PROCEDURE getEmployeesByRange(IN min decimal(10,2) , in max

```

decimal(10,2))

-> BEGIN

-> select * from employees where salary between min and max;

-> END //

Query OK, 0 rows affected (0.01 sec)

mysql> DELIMITER ;

mysql> call getEmployeesByRange(40000,80000);

emp_id	emp_name	department	salary	experience	city	age
101	Ravi	IT	45000.00	2	Hyderabad	24
102	Kiran	HR	40000.00	3	Vijayawada	27
103	Suresh	IT	55000.00	5	Hyderabad	30
105	Priya	Finance	60000.00	6	Bangalore	32
106	Sneha	IT	50000.00	4	Vizag	28
107	Rahul	HR	42000.00	2	Hyderabad	25
109	Divya	Finance	65000.00	7	Bangalore	34
110	Manoj	IT	70000.00	8	Hyderabad	35

8 rows in set (0.00 sec)

Query OK, 0 rows affected (0.02 sec)

mysql> call getEmployeesByRange(50000,80000);

emp_id	emp_name	department	salary	experience	city	age
103	Suresh	IT	55000.00	5	Hyderabad	30
105	Priya	Finance	60000.00	6	Bangalore	32
106	Sneha	IT	50000.00	4	Vizag	28
109	Divya	Finance	65000.00	7	Bangalore	34
110	Manoj	IT	70000.00	8	Hyderabad	35

5 rows in set (0.00 sec)

Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER //

mysql> CREATE PROCEDURE getDepartmentByexperience(in dept varchar(20), in exp int)

-> BEGIN

-> select * from employees where department=dept and experience>=exp;

-> END //

Query OK, 0 rows affected (0.01 sec)

mysql> DELIMITER ;

mysql> call getDepartmentByexperience('IT',5);

emp_id	emp_name	department	salary	experience	city	age
103	Suresh	IT	55000.00	5	Hyderabad	30

103	Suresh	IT	55000.00	5	Hyderabad	30
110	Manoj	IT	70000.00	8	Hyderabad	35

2 rows in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)

mysql> call getDepartmentByexperience('Finance',5);

emp_id	emp_name	department	salary	experience	city	age
105	Priya	Finance	60000.00	6	Bangalore	32
109	Divya	Finance	65000.00	7	Bangalore	34

2 rows in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)

mysql> Show procedure status
-> where db = database();

Db	Name	Type	Definer	Modified
Created	Security_type	Comment	character_set_client	
collation_connection	Database Collation			
companydb	getDepartmentByexperience	PROCEDURE	root@localhost	2026-09-23 15:00:19
2026-09-23 15:00:19	DEFINER		cp850	
cp850_general_ci	utf8mb4_0900_ai_ci			
companydb	getEmployeeById	PROCEDURE	root@localhost	2026-09-23 14:35:18
2026-09-23 14:35:18	DEFINER		cp850	
cp850_general_ci	utf8mb4_0900_ai_ci			
companydb	getEmployeeBysal	PROCEDURE	root@localhost	2026-09-23 14:40:28
2026-09-23 14:40:28	DEFINER		cp850	
cp850_general_ci	utf8mb4_0900_ai_ci			
companydb	getEmployeesBydepartment	PROCEDURE	root@localhost	2026-09-23 14:46:27
2026-09-23 14:46:27	DEFINER		cp850	
cp850_general_ci	utf8mb4_0900_ai_ci			
companydb	getEmployeesByRange	PROCEDURE	root@localhost	2026-09-23 14:53:49
2026-09-23 14:53:49	DEFINER		cp850	
cp850_general_ci	utf8mb4_0900_ai_ci			

5 rows in set (0.00 sec)

mysql>

```
mysql>
mysql> /* OUT Parameter
/*> -----
/*> An Out parameter is used to return a value from the stored procedure to the
caller.
/*> the caller provides a variable to receive the result..
/*>
/*> Syntax
/*> -----
/*> DELIMITER //
/*> CREATE PROCEDURE PROCEDURE_NAME(OUT parameter_name datatype)
/*> BEGIN
/*>     SET parameter_name=value;
/*> END // */
```

```
mysql>
mysql> DELIMITER //
mysql> CREATE PROCEDURE Total_emp(OUT total Int)
-> BEGIN
->     select count(*) into total from employees;
-> END //
```

Query OK, 0 rows affected (0.01 sec)

```
mysql> DELIMITER ;
mysql> set @total=0;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> DELIMITER ;
mysql> set @total=0;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> call Total_emp(@total);
Query OK, 1 row affected (0.02 sec)
```

```
mysql> select @total;
+-----+
| @total |
+-----+
|      10 |
+-----+
1 row in set (0.00 sec)
```

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE max_salary(OUT max_sal decimal(10,2))
-> BEGIN
->     select max(salary) into max_sal from employees;
-> END //
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> DELIMITER ;
```

```
mysql> set @max_sal=0;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> call max_salary(@max_sal);
Query OK, 1 row affected (0.00 sec)
```

```
mysql> select @max_sal;
```

```
+-----+
| @max_sal |
+-----+
| 70000.00 |
+-----+
```

```
1 row in set (0.00 sec)
```

```
mysql> DELIMITER //
```

```
mysql> CREATE PROCEDURE it_emp(out count_it int)
```

```
-> BEGIN
```

```
->     select count(*) into count_it from employees where department='IT';
```

```
-> END //
```

```
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> DELIMITER ;
```

```
mysql> set @count=0;
```

```
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> call it_emp(@count);
```

```
Query OK, 1 row affected (0.00 sec)
```

```
mysql> select @count;
```

```
+-----+
| @count |
+-----+
|      4 |
+-----+
```

```
1 row in set (0.00 sec)
```

```
mysql> show tables;
```

```
+-----+
| Tables_in_companydb |
+-----+
| employees            |
+-----+
```

```
1 row in set (0.00 sec)
```

```
/*INOUT Parameter
```

```
/*> -----
```

```
/*> An INOUT parameter is used when we want to send a value into the procedure
and recieve the modified value back
```

```
/*>
```

```
/*> INOUT=INPUT+OUTPUT
```

```
/*>
```

```

/*> Syntax
/*> -----
/*> CREATE PROCEDURE procedure_name(INOUT parameter_name datatype)
/*> BEGIN
/*>     SET parameter_name=new_value;
/*> END //

```

```

mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE INC_SALARY(
->     IN EID INT,
->     INOUT SALARY_AMOUNT DECIMAL(10,2)
-> )
-> BEGIN
->     SELECT SALARY + SALARY_AMOUNT
->     INTO SALARY_AMOUNT
->     FROM EMPLOYEES
->     WHERE EMP_ID = EID;
-> END //

```

Query OK, 0 rows affected (0.01 sec)

```

mysql>
mysql> DELIMITER ;
mysql> SET @amount=500;
Query OK, 0 rows affected (0.00 sec)

```

```

mysql> call INC_SALARY(101,@amount);
Query OK, 1 row affected (0.00 sec)

```

```

mysql> select @amount;
+-----+
| @amount |
+-----+
| 45500.00 |
+-----+
1 row in set (0.00 sec)

```

```

mysql> DELIMITER //
mysql> CREATE PROCEDURE INC_SALARY(
->     IN EID INT,
->     INOUT SALARY_AMOUNT DECIMAL(10,2)
-> )
-> BEGIN
->     SELECT SALARY + (SALARY_AMOUNT+0.10)
->     INTO SALARY_AMOUNT
->     FROM EMPLOYEES
->     WHERE EMP_ID=EID;
-> END //
mysql>
mysql> DELIMITER //

```

```
mysql> CREATE PROCEDURE INC_SAL(
->     IN EID INT,
->     INOUT SALARY_AMOUNT DECIMAL(10,2)
-> )
-> BEGIN
->     SELECT SALARY + (SALARY_AMOUNT+0.10)
->     INTO SALARY_AMOUNT
->     FROM EMPLOYEES
->     WHERE EMP_ID=EID;
-> END //
```

Query OK, 0 rows affected (0.01 sec)

```
mysql> DELIMITER ;
mysql> set @amount=5000;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> call INC_SAL(101,@amount);
Query OK, 1 row affected (0.00 sec)
```

```
mysql> select @amount;
+-----+
| @amount |
+-----+
| 50000.10 |
+-----+
1 row in set (0.00 sec)
```

```
mysql> drop PROCEDURE INC_SAL;
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE INC_SAL(
->     IN EID INT,
->     INOUT SALARY_AMOUNT DECIMAL(10,2)
-> )
-> BEGIN
->     SELECT SALARY + (SALARY_AMOUNT*0.10)
->     INTO SALARY_AMOUNT
->     FROM EMPLOYEES
->     WHERE EMP_ID=EID;
-> END //
```

Query OK, 0 rows affected (0.01 sec)

```
mysql> DELIMITER ;
mysql> set @amount=6000;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> call INC_SAL(101,@amount);
Query OK, 1 row affected (0.00 sec)
```

```
mysql> select @amount;
```

```
+-----+  
| @amount |
```

```
+-----+  
| 45600.00 |
```

```
+-----+
```

```
1 row in set (0.00 sec)
```

```
mysql> call INC_SAL(102,@amount);  
Query OK, 1 row affected (0.00 sec)
```

```
mysql> select @amount;
```

```
+-----+  
| @amount |
```

```
+-----+  
| 44560.00 |
```

```
+-----+
```

```
1 row in set (0.00 sec)
```

```
mysql> notee
```